Closures in JavaScript

👨‍💼 **Interviewer:**

"Suppose you have a `for` loop that creates buttons inside a web page. You want each button to show its index when clicked. But for some reason, all buttons are showing the same number. Can you explain why this is happening and how closures can help fix this?"

```
for (var i = 0; i < 5; i++) {
  const button = document.createElement('button');
  button.innerText = `Button ${i}`;
  button.onclick = function () {
    console.log(i); // prints 5 every time
  };70
Line Numbers
Support 10015.io
for (var i = 0; i < 5; i++) {
  const button = document.createElement('button');
  button.innerText = `Button ${i}`;
  button.onclick = function () {
    console.log(i); // prints 5 every time
  };
  document.body.appendChild(button);
}

  document.body.appendChild(button);
}
```

👨‍💻 **Candidate (Using What, Why, How):**

🔹 **What is a Closure?**
Closure is when an inner function remembers and has access to variables from its outer function scope, even after the outer function has finished execution.

🔹 **Why does it happen?**
Because in JavaScript, functions form closures. So they retain references to the variables in their lexical scope — not copies. That's why all buttons end up showing the **last value** of the loop variable (e.g., 5 if loop ran 5 times).

🔹 **How do I fix it?**
Use an **IIFE (Immediately Invoked Function Expression)** or `let` instead of `var` to create a new lexical environment for each iteration.

✅ **Fix Using Closure (IIFE):**

```
for (var i = 0; i < 5; i++) {
  (function(index) {
    const button = document.createElement('button');
    button.innerText = `Button ${index}`;
    button.onclick = function () {
      console.log(index); // prints correct index
    };
    document.body.appendChild(button);
  })(i);
}
```

✅ **Modern Fix Using `let`:**

```
for (let i = 0; i < 5; i++) {
  const button = document.createElement('button');
  button.innerText = `Button ${i}`;
  button.onclick = function () {
    console.log(i); // prints correct index
  };
  document.body.appendChild(button);
}
```

Follow-Up Closure-Based Interview Questions

1. **Can you explain how closures are different from scope?**

2. **What will be the output of this and why?**

```
function outer() {
  let count = 0;
  return function () {
    count++;
    console.log(count);
  };
}
const counter1 = outer();
counter1();
counter1();
const counter2 = outer();
counter2();
```

3. **How can closures lead to memory leaks?**

4. **What is the relationship between closures and data privacy in JavaScript?**

5. **How can closures be used to create a once() function (function that only runs once)?**

6. **Write a function using closure that mimics a private variable.**

7. **What are real-world use cases of closures in frontend development?**

8. **Can you use closures to debounce a function?**

9. **Are closures synchronous or asynchronous in nature?**

10. **How do closures behave in asynchronous functions or setTimeout callbacks?**

mycodeshala